

DengueCAD

Methods Guide: imputation and prediction methods compared with NM
Reference and learning guide · October 2026

What this guide is: DengueCAD compares the New Method (NM) of Rao & Kumar [1] against well-known alternatives. NM has two parts: **NMI**, which fills in missing values, and **NMPrediction**, which picks the most useful features and predicts the diagnosis. This guide explains, in plain language, every method NM is compared with, plus NM itself.

How to read each method page

- **In one sentence** — the core idea (yellow boxes mark the NM methods).
- **How it works** — numbered steps, followed by a flowchart of the same steps.
- **A small example** — a tiny worked case with made-up numbers.
- **Settings used in DengueCAD** — the exact configuration used in our experiments.
- **Strengths / Watch out for** — when the method shines and when to be careful.
- **Learn more** — numbered references, listed with links at the end of the guide.

Contents

Section	Methods
1. Background	Missing values, the two-step pipeline, key terms
2. Imputation methods	2.1 NMI · 2.2 MICE · 2.3 kNN · 2.4 MissForest · 2.5 Mean / Median / Mode
3. Prediction methods	3.1 NMPrediction (with GA wrapper and ADT) · 3.2 C4.5 · 3.3 Logistic regression · 3.4 SVM
4. Side-by-side summary	Quick comparison tables
5. How the methods are scored	Imputation accuracy, Accuracy, SE, SP, AUC
6. References	Papers and documentation with links

1. Background

1.1 Why missing values matter

Clinical records are often incomplete: a lab test was not ordered, a symptom was not recorded, or a form field was left blank. Most prediction methods need a complete table, so the missing cells must be handled first. Dropping incomplete records wastes data and can bias the result; **imputation** instead fills each missing cell with a sensible estimate.

Statisticians describe *why* values are missing with three labels [2, 9]. **MCAR** (missing completely at random): missingness has nothing to do with the data — like a sample tube dropped by accident. **MAR** (missing at random): missingness depends on other recorded values — e.g. older patients skip a test more often. **MNAR** (missing not at random): missingness depends on the missing value itself — e.g. very sick patients miss a follow-up. DengueCAD's controlled experiments hide values MCAR, so we know the true answer and can score each method.

1.2 The two-step pipeline

Every experiment follows the same two steps. First an **imputation** method completes the data; then a **prediction** method (a classifier) learns to predict the decision column — for dengue, whether the patient has the disease.

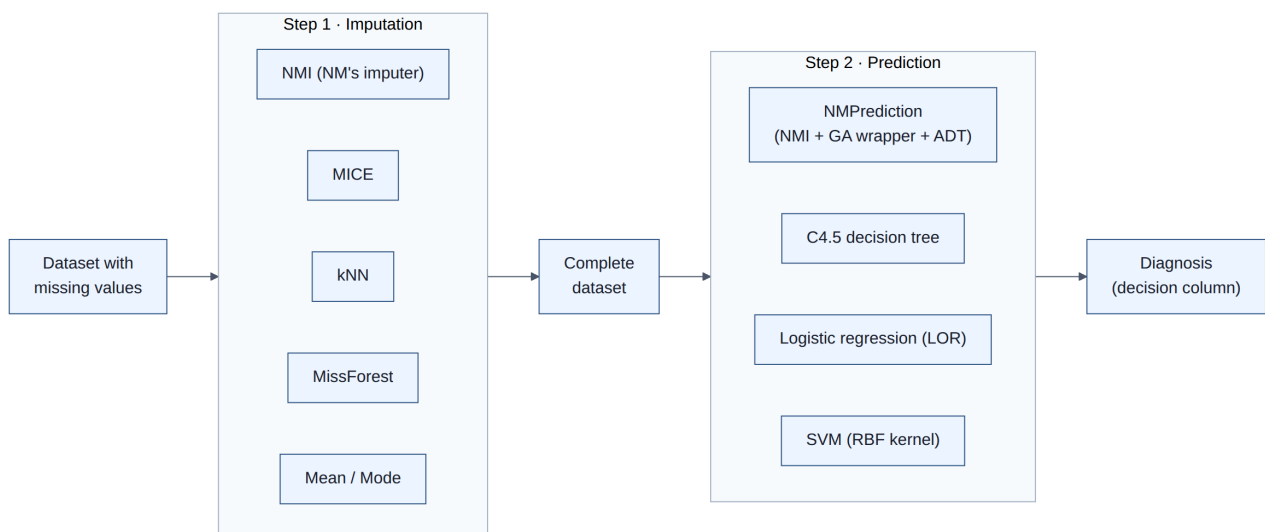


Figure 1. Imputation methods (left) and prediction methods (right) compared in DengueCAD. NM's components are NMI and NMPrediction.

1.3 Key terms

Term	Meaning
Record / row	One patient.
Attribute / feature	One column describing the patient, e.g. fever, platelet count.
Decision column	The diagnosis we want to predict (the last column), e.g. dengue yes / no.
Categorical attribute	Takes values from a fixed set (yes / no, blood group). Averages make no sense, so the most frequent value (mode) is used.

Term	Meaning
Numeric attribute	A measured number (temperature, platelets). Averages (mean) make sense.
Complete record	A row with no missing values. NMI uses these as donors.
Donor / neighbour	A similar complete record whose values are borrowed to fill a gap.
Cross-validation (CV)	Split the data into k parts; train on $k-1$, test on the remaining part, rotate, and average. Gives an honest estimate of performance.
Overfitting	A model memorises the training data and does worse on new patients.

2. Imputation methods

Each method below takes a table with gaps and returns a complete table. They differ in how they decide what value is "sensible": a single column summary (Mean / Mode), similar patients (NMI, kNN), or a model learned from the other columns (MICE, MissForest).

2.1 NMI — New Method Imputation (NM's imputer)

In one sentence: fill each gap with the most common value (or the average) among complete records that are closer than average to the incomplete record, where closeness also takes the diagnosis into account.

How it works

- 1 Separate the complete records (**donors**) from the incomplete ones (**targets**). Records whose diagnosis is missing are left alone.
- 2 For a target record, compare it with every donor, one attribute at a time, using a score that depends on whether the two records share the same diagnosis (see the four cases below).
- 3 Combine the attribute scores into a single distance, and convert all distances to z-scores.
- 4 Every donor with a z-score ≤ 0 (closer than average) is a **neighbour**.
- 5 Fill each missing categorical / integer value with the neighbours' **mode**, and each missing real value with the neighbours' **mean**. Repeat for every target record.

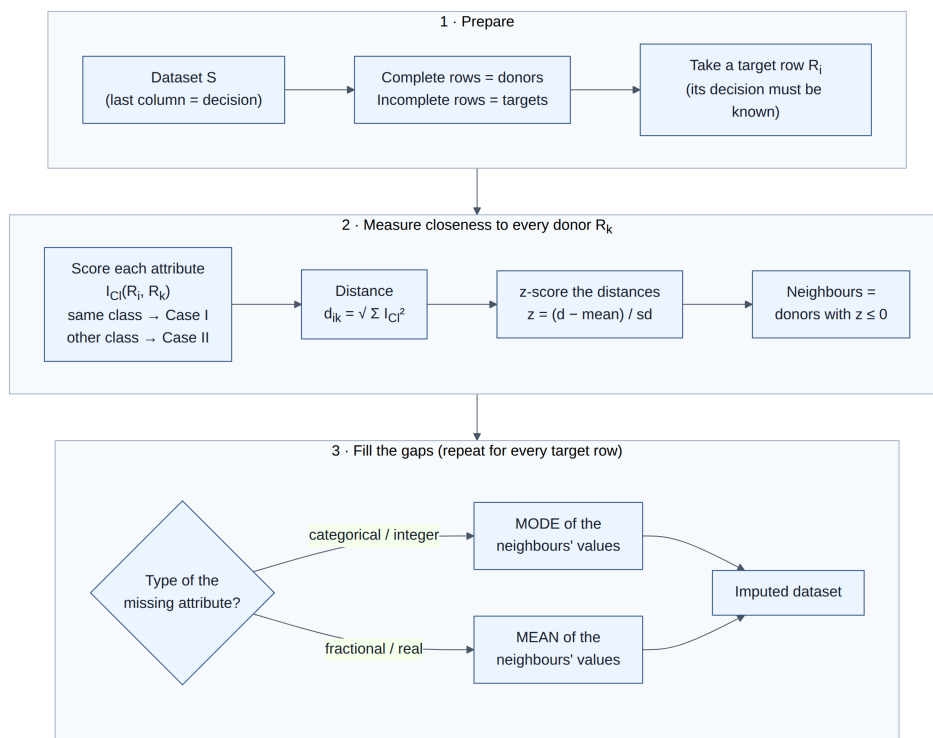


Figure 2. NMI: prepare donors and targets, measure closeness, fill the gaps.

The four scoring cases

For a target row R_i and a donor R_k , each attribute C_l gets a score $I_{C_l}(R_i, R_k)$. Which formula applies depends on whether the two patients share the same diagnosis and on the attribute's type:

	Categorical / integer attribute	Fractional / real attribute
Case I same decision class	$I = \min(\gamma_i / \gamma_k, \gamma_k / \gamma_i)$ γ = how often each record's value occurs among complete records of that class	$I = \min(A_{il} / A\#, A_{kl} / A\#)$ $A\#$ = mean of column l
Case II different classes	$I = \max(\beta / \delta, \delta / \beta)$ β, δ = how often each value occurs within its own record's class	$I = \max(A_{il} / \Lambda, A_{kl} / \Lambda)$ $\Lambda = \min(P\#, Q\#)$, the smaller of the two class means of column l

The scores are combined into one distance, which is standardised so that “close” means “closer than average”:

$$d_{ik} = \sqrt{(\sum_l I_{Cl}(R_i, R_k))^2}$$

$$z(d_{ik}) = (d_{ik} - \text{mean}(d)) / \text{sd}(d)$$

$$\text{neighbours} = \{ R_k : z(d_{ik}) \leq 0 \}$$

Because the neighbour rule is “below-average distance”, NMI does not need a k to be chosen; about half of the donors usually qualify. If none do, the single closest donor is used.

A small example

A dengue patient has *fever* and *platelets* missing. Six complete records are donors, with distances 0.8, 1.0, 1.2, 2.5, 2.9 and 3.1 (mean 1.92). The first three are below the mean ($z \leq 0$), so they are the neighbours. Their fever values are yes, yes, no → **mode = yes**. Their platelet counts are 90k, 110k, 100k → **mean = 100k**.

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	nmilib.non_parametric_imputation (sibling NMI repository)
Donors	Complete records only; rows with a missing decision are not imputed
Neighbours	All donors with $z(d) \leq 0$ (no k to tune)
Fill rule	Mode for categorical / integer, mean for fractional / real
Hold-out experiment	All attributes ordinal-encoded, so the mode is used everywhere

Strengths	Watch out for
• Works on mixed categorical and numeric data. • Uses the diagnosis when measuring similarity, so neighbours are clinically relevant. • No parameters to tune (no k, no model). • Easy to explain: values come from real, similar patients.	• Needs enough complete records; with many gaps, few donors remain. • Cannot impute records whose diagnosis is missing. • Comparing every target with every donor is slow on very large data. • Taking the mode/mean of ~half the donors can pull values towards the class average.

Learn more: [1]

2.2 MICE — Multivariate Imputation by Chained Equations

In one sentence: predict each incomplete column from all the other columns with a regression model, and cycle through the columns several times until the filled-in values stop changing.

How it works

- 1 Fill every gap temporarily with the column mean.
- 2 Pick a column that had missing values. Fit a regression model that predicts it from all the other columns, using only the rows where it was observed.
- 3 Replace that column's missing cells with the model's predictions.
- 4 Move to the next incomplete column and repeat. One pass over all columns is a **round**.
- 5 Run more rounds until the imputed values settle (or a maximum is reached).

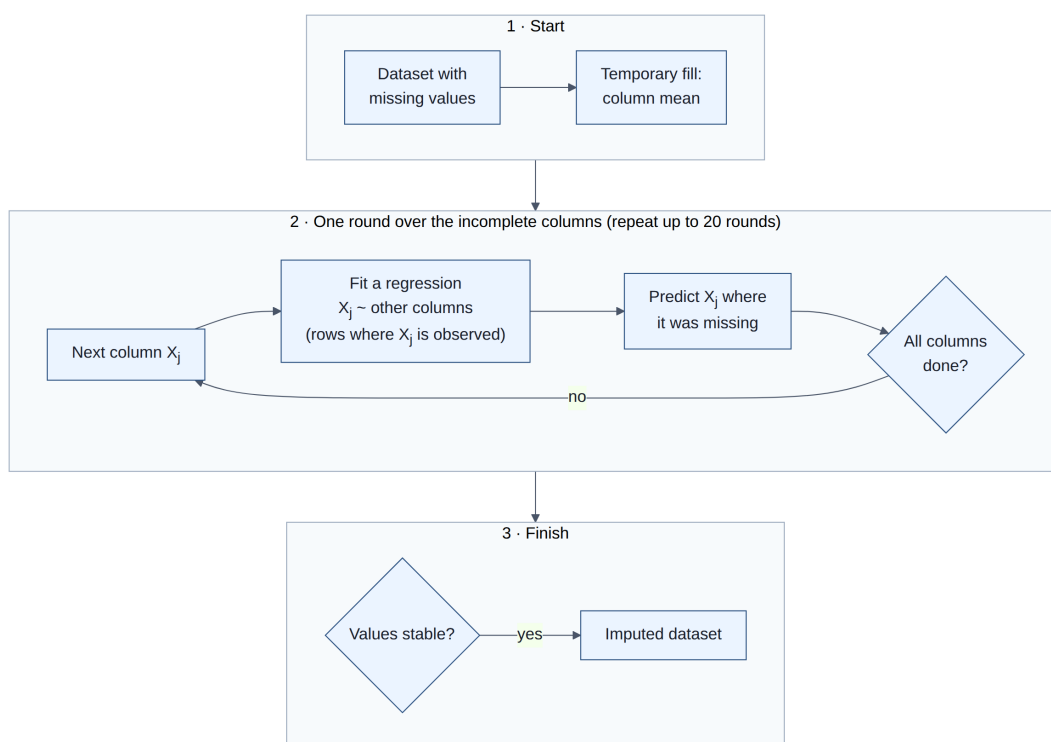


Figure 3. MICE: chained regressions, one column at a time, repeated in rounds.

A small example

Platelets is missing for some patients. MICE fits $\text{platelets} \approx b_0 + b_1 \cdot \text{WBC} + b_2 \cdot \text{age} + \dots$ on the patients whose platelets are known, then uses the fitted equation to predict the missing platelet counts. Next it does the same for *WBC* using the newly filled platelets, and so on.

In full multiple imputation, MICE builds several completed datasets with random draws and pools the results [3]. DengueCAD uses the single, deterministic version.

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	scikit-learn IterativeImputer [4]
Model per column	Bayesian ridge regression (scikit-learn default)
Initial fill	Column mean
Rounds	Up to 20 (max_iter = 20)
Draws	sample_posterior = False → one deterministic imputation
Categorical data	Predictions are rounded to the nearest valid code

Strengths	Watch out for
• Uses relationships between columns, not just column averages. • Very widely used and well studied in medical statistics. • Flexible: any regression model can be plugged in.	• Default linear models miss non-linear patterns and interactions. • Treats category codes as numbers unless a classifier is used per column. • Several rounds over all columns can be slow on wide data.

Learn more: [3], [4]

2.3 kNN imputation — k-Nearest Neighbours

In one sentence: find the k most similar records that have the value, and fill the gap with their (distance-weighted) average.

How it works

- 1 For a record with a missing value in column X, compute its distance to every other record, using only the columns both records have (nan-Euclidean distance).
- 2 Keep the k = 5 closest records whose X is known.
- 3 Give closer neighbours more say: weight each by 1 / distance.
- 4 Fill X with the weighted average of the neighbours' values.

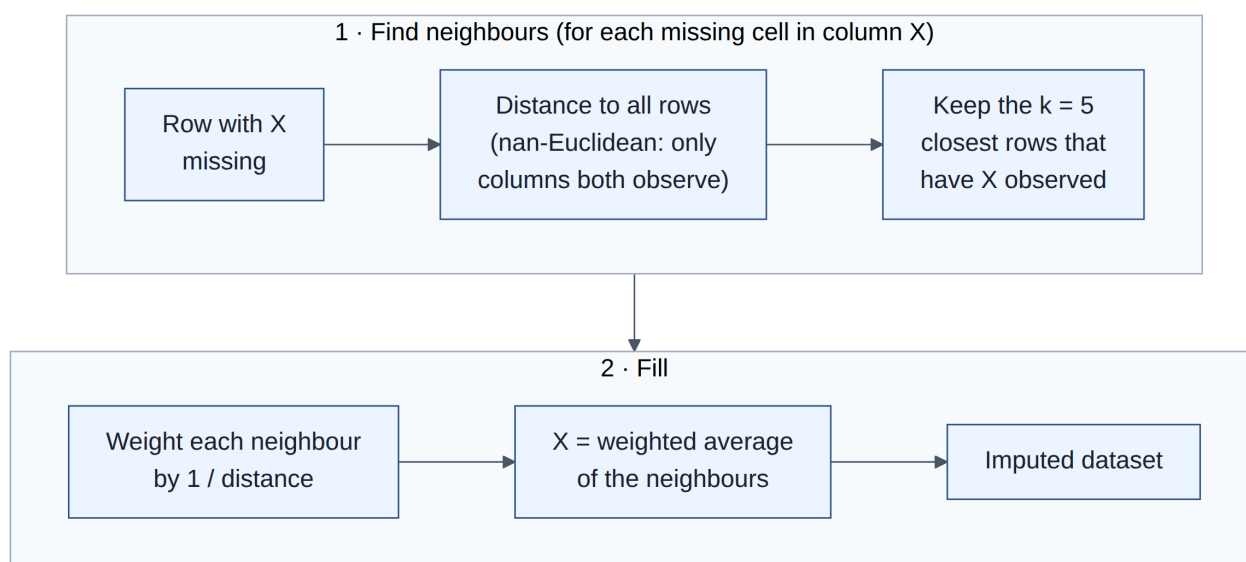


Figure 4. kNN imputation: find the k closest records, then take a weighted average.

A small example

A patient's *haemoglobin* is missing. Their five closest patients have 12, 13, 13, 14 and 15 g/dL at distances 1, 1, 2, 2, 4. Weights are 1, 1, 0.5, 0.5, 0.25, giving $(12 + 13 + 6.5 + 7 + 3.75) / 3.25 \approx \mathbf{13.0 \text{ g/dL}}$.

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	scikit-learn KNNImputer [6], after Troyanskaya et al. [5]
Neighbours	$k = 5$
Weights	$1 / \text{distance}$ (weights = "distance")
Distance	nan-Euclidean (ignores columns missing in either record)
Categorical data	Averages are rounded to the nearest valid code

Strengths	Watch out for
- Simple and intuitive: borrow from similar patients. - No model to fit; captures local, non-linear structure. - Can use incomplete records as neighbours.	k must be chosen; results depend on it. - Distances are sensitive to feature scaling and irrelevant columns. - Ignores the diagnosis; averaging categories needs rounding. - Slow for large datasets (distance to every row).

Learn more: [5], [6]

2.4 MissForest

In one sentence: like MICE, but each column is predicted by a random forest, which handles mixed categorical / numeric data and non-linear relationships.

How it works

- 1 Fill gaps temporarily: mean for numeric columns, mode for categorical columns.
- 2 Order the columns from fewest to most missing values.
- 3 For each column, train a **random forest** (many decision trees on random subsets of rows and features [8]) on the rows where the column is observed, and predict the missing cells — a classification forest for categories, a regression forest for numbers.
- 4 After all columns, compare the new table with the previous round.
- 5 Stop when the change is tiny (or a maximum number of rounds is reached).

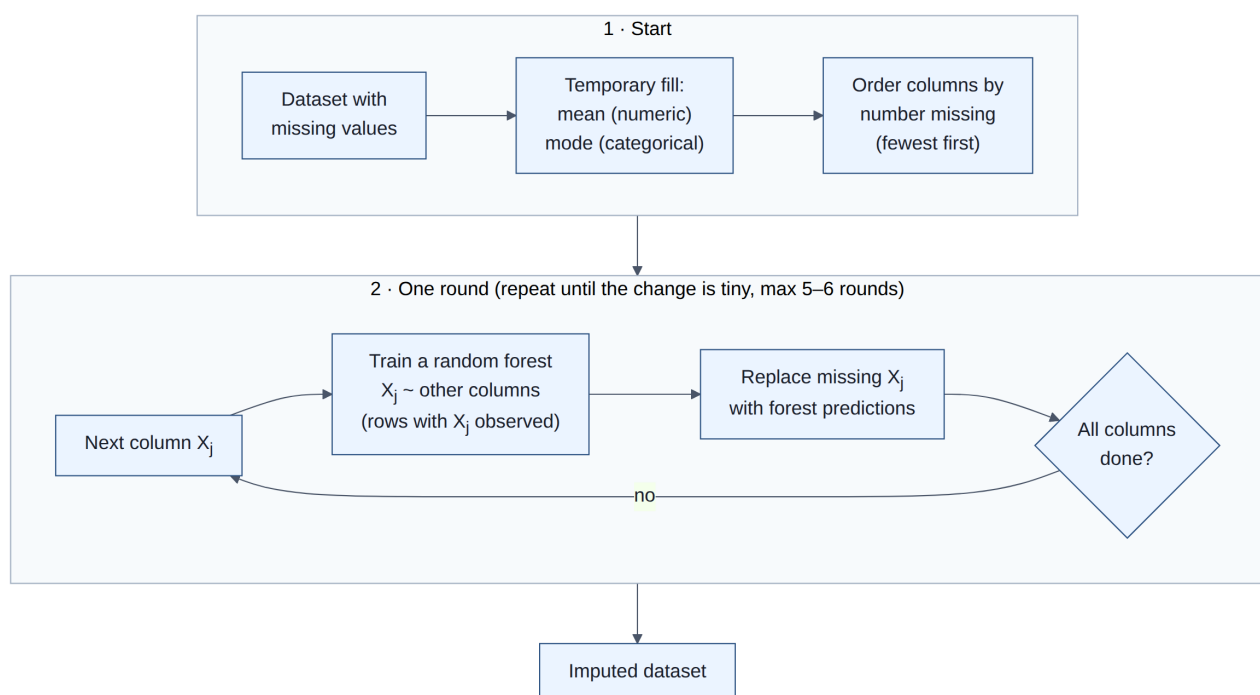


Figure 5. MissForest: iterative imputation with a random forest per column.

A small example

Rash (yes / no) is missing for some patients. MissForest trains a classification forest that predicts rash from fever, platelets, age and the other columns. If most trees vote "yes" for a patient, the gap is filled with yes. Numeric columns such as platelets are filled with the average prediction of the trees.

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	denguecad/missforest_impute.py (follows Stekhoven & Bühlmann [7])
Forest size	40 trees per column
Rounds	Up to 6 (benchmark) / 5 (hold-out experiment)
Stopping rule	Mean squared change between rounds $< 10^{-6}$
Categorical columns	Random forest classifier on integer codes

Strengths	Watch out for
• Handles mixed data types and complex, non-linear relationships. • No assumptions about the data distribution. • Often among the most accurate imputers in published comparisons [7].	• Much slower than the other methods (many forests per round). • Less transparent: hard to explain why a value was chosen. • Can overfit small datasets.

Learn more: [7], [8]

2.5 Mean / Median / Mode imputation

In one sentence: fill every gap in a column with one summary value of that column — the mean, the median or the most frequent value.

How it works

- 1 For each column, compute one summary of the observed values: the **mean** (numeric), the **median** (numeric and skewed) or the **mode** (categorical).
- 2 Write that value into every missing cell of the column.

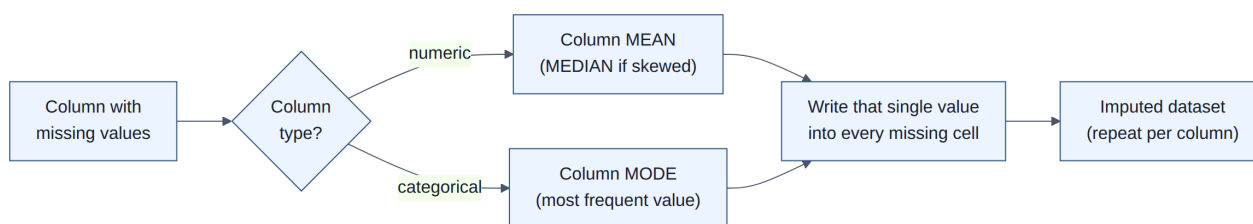


Figure 6. Mean / Median / Mode: one value per column.

A small example

Fever is recorded as yes, yes, no, yes for four patients and missing for a fifth: the mode is **yes**. Age values 20, 30, 40 and 90: the mean is 45, the median is 35 — the median is less affected by the unusual 90.

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	scikit-learn SimpleImputer [10]
Imputation benchmark	Mean, median, or mode (mode for categorical + mean for numeric)
Hold-out experiment	Mode on the ordinal-encoded attributes

Strengths	Watch out for
• Extremely fast and simple; a standard baseline [9]. • Never fails, even with few complete records.	• Ignores every other column and the diagnosis. • Shrinks variability and weakens relationships between columns. • Every patient with a gap gets the same value.

Learn more: [9], [10]

3. Prediction methods

A prediction method (classifier) learns from patients whose diagnosis is known and then predicts the diagnosis of new patients. NMPrediction is NM's full pipeline; C4.5, logistic regression and SVM are the standard classifiers it is compared with in the paper [1].

3.1 NMPrediction — NM's prediction pipeline (Algorithm 1)

In one sentence: impute with NMI, let a genetic algorithm choose the most influential features, then predict with a boosted Alternating Decision Tree.

How it works

- 1 **Impute** the training data with NMI (Section 2.1).
- 2 **Select features** with a GA wrapper: the genetic algorithm searches for the subset of attributes on which the classifier is most accurate.
- 3 **Train an ADT** on the selected features.
- 4 **Evaluate** with stratified 10-fold cross-validation (Accuracy, AUC, sensitivity, specificity), or predict new patients using only the selected features.

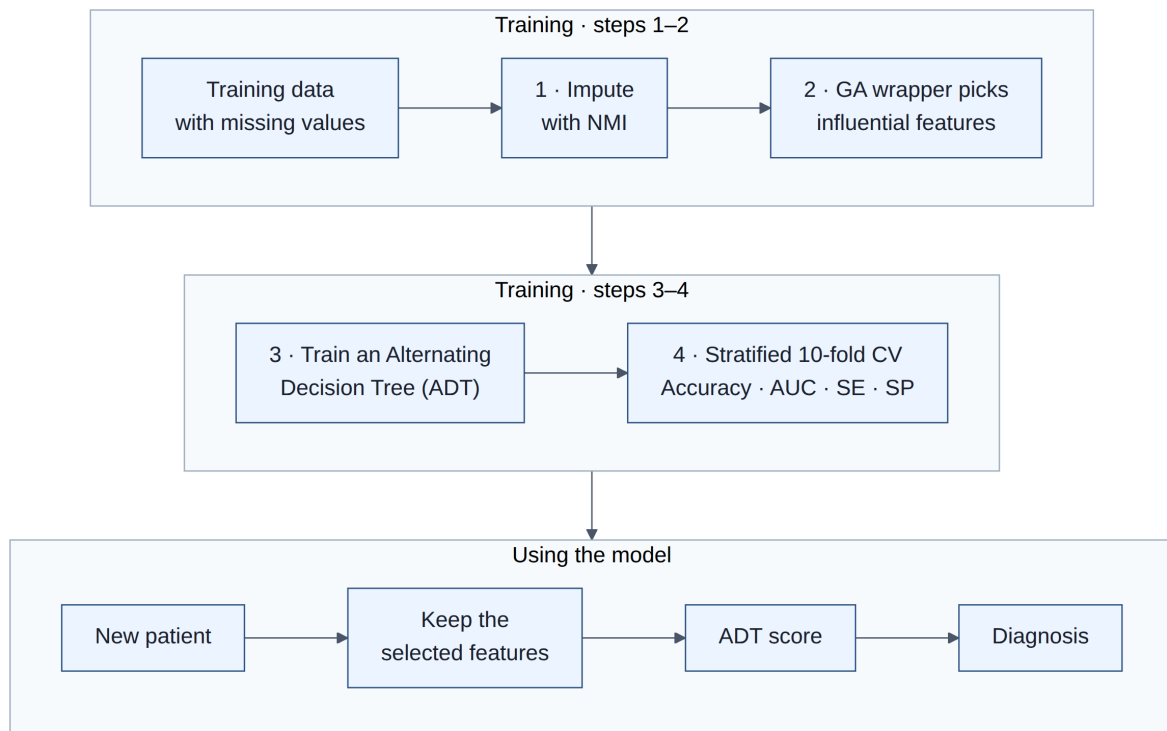


Figure 7. NMPrediction: NMI → GA wrapper → ADT.

Inside NMPrediction (a): the GA wrapper

A **wrapper** judges a feature subset by actually training the classifier on it [11]. Trying every subset is impossible (2^n subsets for n features), so a **genetic algorithm** [12] searches instead: each candidate subset is a string of bits (1 = keep), the best strings survive and are combined, and random bit flips keep exploring.

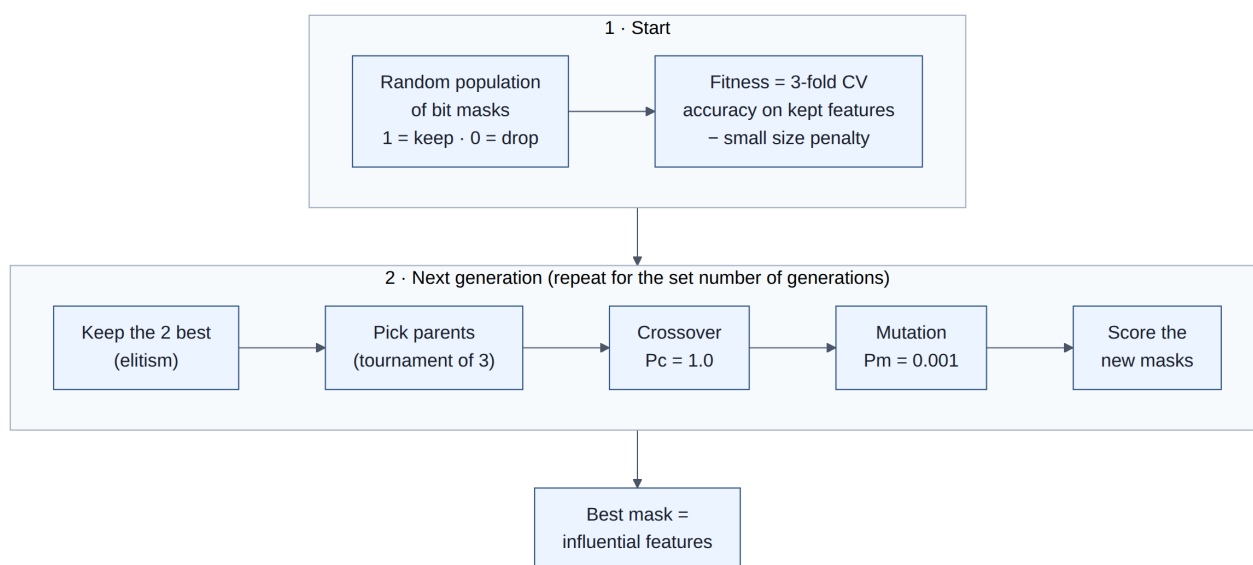


Figure 8. GA wrapper: evolve bit masks scored by cross-validated accuracy.

Inside NMPrediction (b): the Alternating Decision Tree (ADT)

An ADT [13] is built by **boosting** [14]: it adds one simple rule per round, each time focusing on the patients the current model gets wrong. Every rule carries a score for YES and for NO; to classify a patient, the scores of all rules that apply are added up and the sign of the total decides. The sum also shows how confident the prediction is.

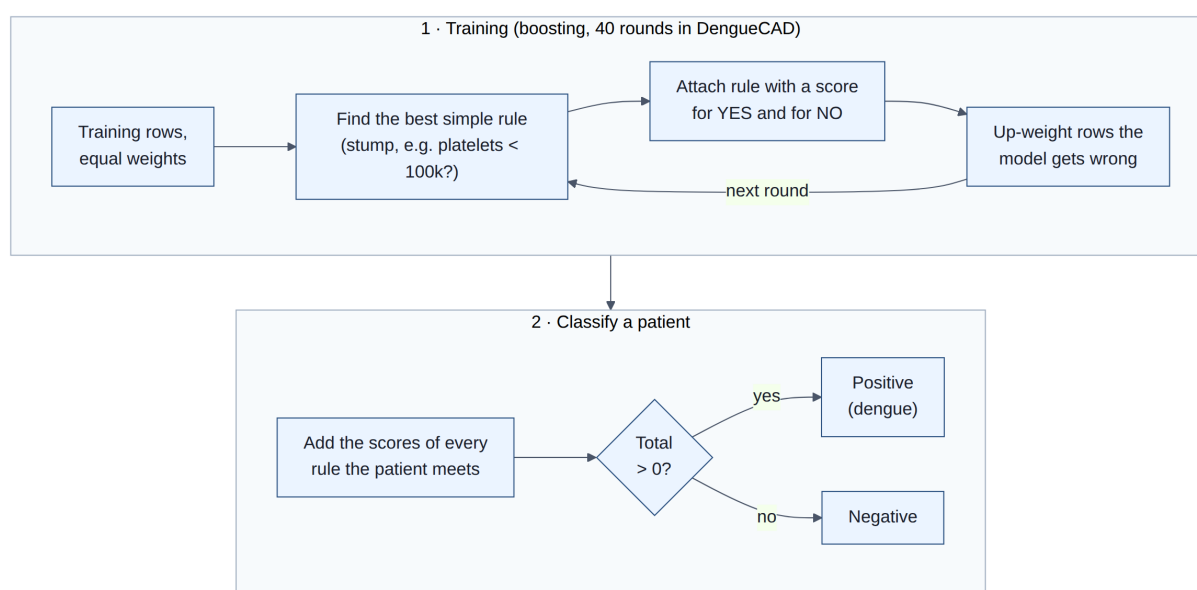


Figure 9. ADT: boosted rules with additive scores.

A small example

From 20 clinical attributes, the GA might keep 6 — say fever duration, platelets, WBC, rash, headache and NS1. The ADT then learns rules such as “platelets < 100k → +0.8, otherwise −0.4”. A new patient's scores are summed; a positive total means dengue. The selected attributes are also reported as the most influential symptoms (Table III).

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	denguecad/nm_prediction.py, feature_selection.py, adt.py
GA operators	Crossover $P_c = 1.0$, mutation $P_m = 0.001$, elitism 2, tournament of 3
GA size	Population 12×8 generations (hold-out experiment: 8×5)
GA fitness	3-fold CV accuracy of the ADT – $0.01 \times$ fraction of features kept
ADT	40 boosting rounds of decision stumps (AdaBoost) with additive scores
Evaluation	Stratified 10-fold CV; hold-out validation / test sets

Strengths	Watch out for
• Handles missing values as part of the pipeline. • Picks a small, interpretable set of influential symptoms. • ADT rules are readable and give a confidence score. • Boosting usually gives strong accuracy from simple rules.	• The GA is the slowest part (many cross-validated fits). • Results vary slightly with the random seed of the GA. • DengueCAD's ADT approximates Freund & Mason's tree with boosted stumps (rules are not nested under one another).

Learn more: [1], [11], [12], [13], [14]

3.2 C4.5 decision tree

In one sentence: ask the most informative yes/no question first, split the patients by the answer, and keep asking questions until each group is (nearly) one diagnosis.

How it works

- 1 Start with all training patients in one node.
- 2 For each attribute, measure how much splitting on it reduces uncertainty about the diagnosis (**information gain**; C4.5 uses the gain *ratio* to avoid favouring attributes with many values [15, 16]).
- 3 Split on the best attribute (or threshold for numbers) and create child nodes.
- 4 Repeat on each child until a node is pure or too small; it becomes a leaf.
- 5 To predict, follow the questions from the root down to a leaf.

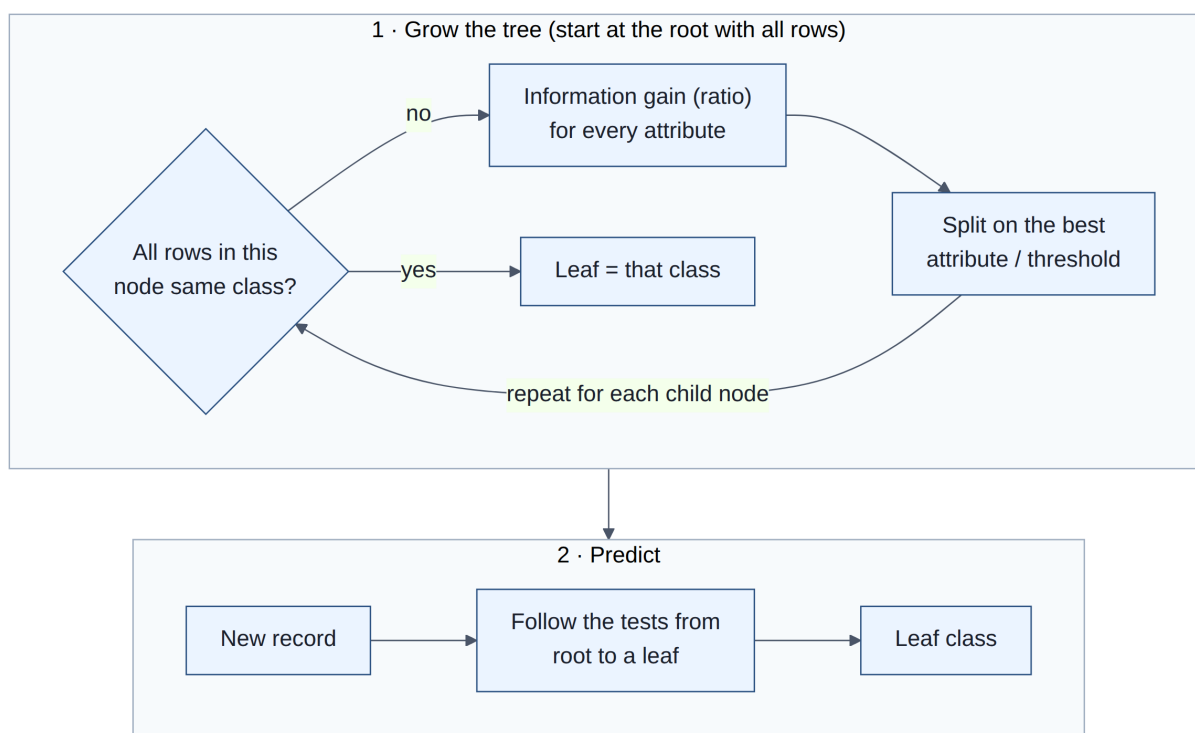


Figure 10. C4.5: grow the tree by repeatedly splitting on the most informative attribute.

A small example

Root question: “platelets < 100k?”. If yes → next question “fever > 3 days?” → leaf **dengue**. If no → leaf **not dengue**. Entropy measures uncertainty: a node with 50 % dengue has entropy 1 bit; a pure node has 0. The question that lowers entropy the most is asked first.

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	scikit-learn DecisionTreeClassifier [17]
Split criterion	Entropy / information gain (criterion = "entropy")
Note	scikit-learn grows binary CART-style trees without C4.5's gain ratio and pruning; this is the usual open-source stand-in for C4.5

Strengths	Watch out for
• Very easy to read and explain to clinicians. • Handles numeric and categorical data; no scaling needed. • Fast to train and to predict.	• A single tree overfits easily (unstable to small data changes). • Axis-aligned splits approximate smooth boundaries poorly. • Usually less accurate than boosted or kernel methods.

Learn more: [15], [16], [17]

3.3 Logistic regression (LOR)

In one sentence: add up the features with learned weights and pass the total through an S-shaped curve to get the probability of disease.

How it works

- 1 Standardise every feature (mean 0, standard deviation 1).
- 2 Compute a score $z = b_0 + b_1x_1 + \dots + b_px_p$.
- 3 Turn the score into a probability with the sigmoid $p = 1 / (1 + e^{-z})$.
- 4 Adjust the weights b to make the true diagnoses as likely as possible (maximum likelihood), with a small penalty on large weights.
- 5 Predict positive when $p \geq 0.5$.

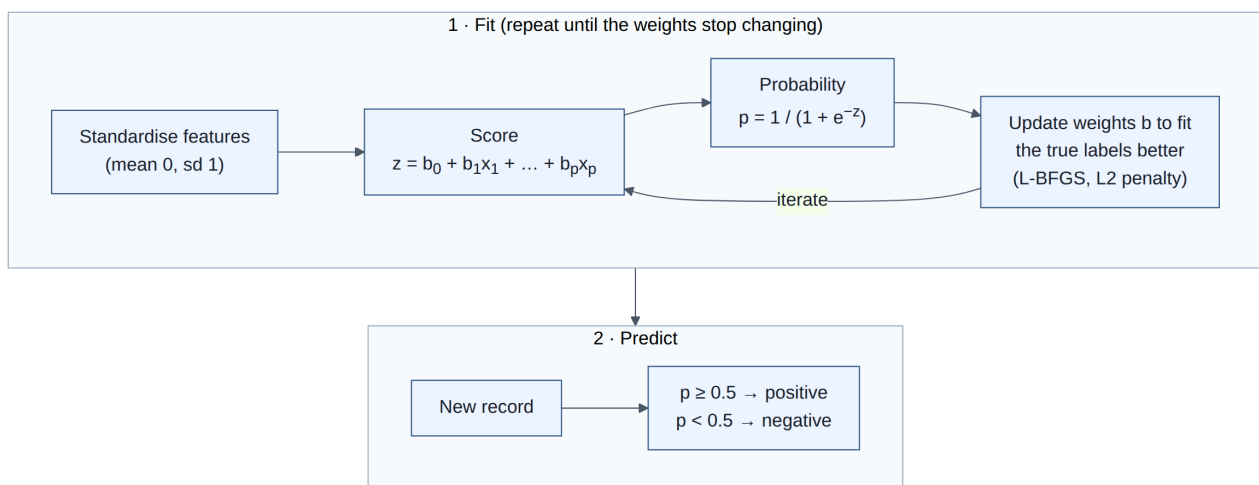


Figure 11. Logistic regression: weighted sum → sigmoid → probability.

A small example

Suppose $z = -1 + 2 \cdot (\text{low platelets}) + 1 \cdot (\text{fever})$. A patient with both gets $z = 2$, so $p = 1 / (1 + e^{-2}) \approx \mathbf{0.88} \rightarrow$ dengue. With neither, $z = -1$ and $p \approx 0.27 \rightarrow$ not dengue. Each weight shows how much a feature raises the odds (e^b is the odds ratio).

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	scikit-learn LogisticRegression [19] after StandardScaler
Solver	L-BFGS, up to 2000 iterations
Penalty	L2 with C = 1.0 (scikit-learn default)

Strengths	Watch out for
• Gives probabilities and interpretable odds ratios [18]. • Fast, stable and hard to overfit with regularisation. • A standard reference model in medicine.	• Boundary is linear in the features; misses interactions unless added by hand. • Sensitive to strongly correlated features. • Needs numeric inputs (categories must be encoded).

Learn more: [18], [19]

3.4 Support Vector Machine (SVM) with RBF kernel

In one sentence: find the boundary that separates the two diagnoses with the widest possible margin, using a kernel so the boundary can curve.

How it works

- 1 Standardise every feature.
- 2 Measure similarity between patients with the RBF (Gaussian) kernel $K(x, x') = \exp(-\gamma \|x - x'\|^2)$: 1 for identical patients, near 0 for very different ones.
- 3 Find the boundary with the widest margin between the classes; the parameter C trades a wider margin against training mistakes [20].
- 4 The training patients on or inside the margin are the **support vectors**; only they define the boundary.
- 5 Predict a new patient from its weighted similarity to the support vectors.

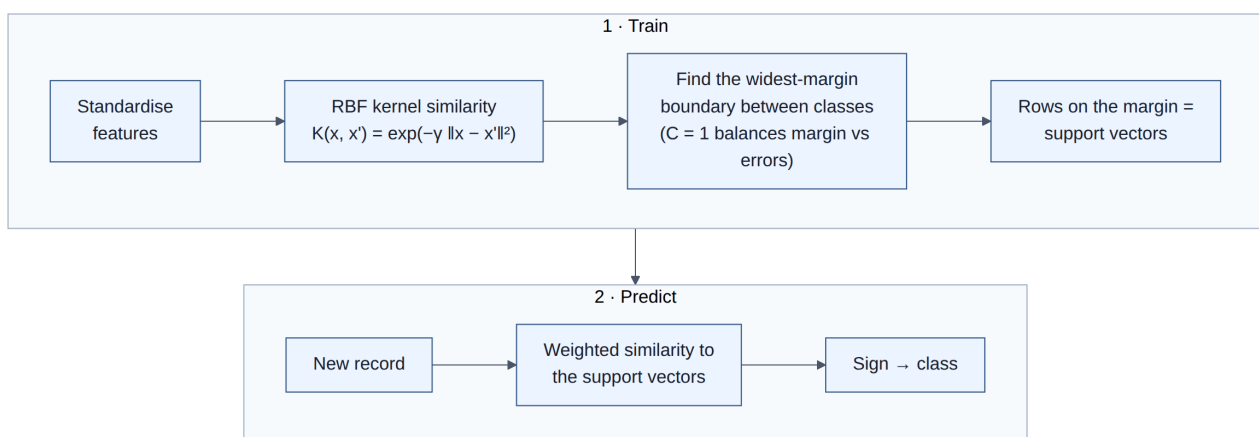


Figure 12. SVM: kernel similarity, widest-margin boundary, support vectors.

A small example

Plot patients by platelets and WBC. A straight line may not separate dengue from non-dengue, but the RBF kernel lets the boundary bend around clusters of dengue patients. A new patient close (in kernel terms) to dengue support vectors is predicted dengue.

Settings used in DengueCAD

Setting	Value used in DengueCAD
Implementation	scikit-learn SVC [22] (LIBSVM [21]) after StandardScaler
Kernel	RBF, $\gamma = \text{"scale"} = 1 / (\text{number of features} \times \text{variance of } X)$
Regularisation	$C = 1.0$
Large data	Training subsampled to 8,000 rows in the hold-out experiment

Strengths	Watch out for
• Often the most accurate of the baselines; flexible, non-linear boundaries. • Works well with many features. • Only the support vectors matter, which resists outliers far from the boundary.	• Training time grows quickly with the number of rows. • A black box: no simple rules or feature weights. • Needs scaling and tuning of C and γ; no native probabilities.

Learn more: [20], [21], [22]

4. Side-by-side summary

4.1 Imputation methods

Method	Core idea	Uses other columns?	Uses diagnosis?	Mixed types	Speed
NMI	Mode / mean of below-average-distance donors	Yes (distance)	Yes	Yes	Medium
MICE	Regression per column, in rounds	Yes (model)	No	Partly	Medium
kNN	Weighted mean of k nearest rows	Yes (distance)	No	Partly	Medium
MissForest	Random forest per column, in rounds	Yes (model)	No	Yes	Slow
Mean / Mode	One value per column	No	No	Yes	Very fast

4.2 Prediction methods

Method	Core idea	Feature selection	Interpretable?	Boundary	Speed
NMPrediction	NMI + GA wrapper + boosted ADT	Yes (GA)	Yes (rules + scores)	Non-linear	Slow (GA)
C4.5	Tree of most informative questions	Implicit	Yes (tree)	Axis-aligned	Fast
LOR	Weighted sum $\rightarrow$ sigmoid	No	Yes (odds ratios)	Linear	Fast
SVM (RBF)	Widest margin with a kernel	No	No	Non-linear	Medium-slow

Yellow rows are NM's components. Speeds are relative, for the dataset sizes used in DengueCAD.

5. How the methods are scored

Imputation accuracy. Start from complete records, hide some values on purpose (MCAR), impute, and count the share of hidden cells restored to their true value. In the hold-out experiment all attributes are ordinal-encoded (numbers binned into deciles), so a cell counts as correct only when the exact code is recovered.

Prediction metrics. With TP / TN = correctly predicted positives / negatives and FP / FN = wrong positives / negatives:

Metric	Formula	Meaning
Accuracy	$(TP + TN) / \text{all}$	Share of patients classified correctly
Sensitivity (SE)	$TP / (TP + FN)$	Share of dengue patients detected
Specificity (SP)	$TN / (TN + FP)$	Share of healthy patients correctly cleared
AUC	Area under the ROC curve	Chance that a random dengue patient is ranked above a random non-dengue patient (0.5 = guessing, 1 = perfect)

Results of these comparisons are in [DengueCAD_Performance_Comparisons.pdf](#) and [DengueCAD_Imputation_Prediction_Holdout.pdf](#) in the same docs folder.

6. References

- [1] Rao VSH, Kumar MN. "A new intelligence-based approach for computer-aided diagnosis of dengue fever." *IEEE Trans. Information Technology in Biomedicine* 16(1):112-118, 2012. <https://doi.org/10.1109/TITB.2011.2171978>
- [2] Rubin DB. "Inference and missing data." *Biometrika* 63(3):581-592, 1976. <https://doi.org/10.1093/biomet/63.3.581>
- [3] van Buuren S, Groothuis-Oudshoorn K. "mice: Multivariate Imputation by Chained Equations in R." *Journal of Statistical Software* 45(3):1-67, 2011. <https://doi.org/10.18637/jss.v045.i03>
- [4] scikit-learn documentation: IterativeImputer. <https://scikit-learn.org/stable/modules/generated/sklearn.impute.IterativeImputer.html>
- [5] Troyanskaya O, et al. "Missing value estimation methods for DNA microarrays." *Bioinformatics* 17(6):520-525, 2001. <https://doi.org/10.1093/bioinformatics/17.6.520>
- [6] scikit-learn documentation: KNNImputer. <https://scikit-learn.org/stable/modules/generated/sklearn.impute.KNNImputer.html>
- [7] Stekhoven DJ, Bühlmann P. "MissForest — non-parametric missing value imputation for mixed-type data." *Bioinformatics* 28(1):112-118, 2012. <https://doi.org/10.1093/bioinformatics/btr597>
- [8] Breiman L. "Random forests." *Machine Learning* 45:5-32, 2001. <https://doi.org/10.1023/A:1010933404324>
- [9] Little RJA, Rubin DB. *Statistical Analysis with Missing Data*, 3rd ed. Wiley, 2019. <https://doi.org/10.1002/9781119482260>
- [10] scikit-learn documentation: SimpleImputer. <https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>
- [11] Kohavi R, John GH. "Wrappers for feature subset selection." *Artificial Intelligence* 97(1-2):273-324, 1997. [https://doi.org/10.1016/S0004-3702\(97\)00043-X](https://doi.org/10.1016/S0004-3702(97)00043-X)
- [12] Goldberg DE. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
- [13] Freund Y, Mason L. "The alternating decision tree learning algorithm." *Proc. 16th ICML*, pp. 124-133, 1999. <https://mlanthology.org/icml/1999/freund1999icml-alternating/>
- [14] Freund Y, Schapire RE. "A decision-theoretic generalization of on-line learning and an application to boosting." *J. Computer and System Sciences* 55(1):119-139, 1997. <https://doi.org/10.1006/jcss.1997.1504>
- [15] Quinlan JR. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993. <https://dl.acm.org/doi/abs/10.5555/152181>
- [16] Quinlan JR. "Induction of decision trees." *Machine Learning* 1:81-106, 1986. <https://doi.org/10.1007/BF00116251>
- [17] scikit-learn user guide: Decision Trees. <https://scikit-learn.org/stable/modules/tree.html>
- [18] Cox DR. "The regression analysis of binary sequences." *J. Royal Statistical Society B* 20(2):215-242, 1958. <https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>
- [19] scikit-learn user guide: Logistic regression. https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- [20] Cortes C, Vapnik V. "Support-vector networks." *Machine Learning* 20:273-297, 1995. <https://doi.org/10.1007/BF00994018>
- [21] Chang C-C, Lin C-J. "LIBSVM: a library for support vector machines." *ACM Trans. Intelligent Systems and Technology* 2(3):27, 2011. <https://doi.org/10.1145/1961189.1961199>
- [22] scikit-learn user guide: Support Vector Machines. <https://scikit-learn.org/stable/modules/svm.html>